

PYTHON ASSIGNMENT – L2

1. Problem Statement

A State electricity distribution utility company owns several thousand physical assets across its supply area in Bhubaneswar City. These includes: distribution poles, distribution transformer, UG cable maintenance chambers. While taking maintenance decisions on the utility infrastructures the utility company requires information on where each asset is, it's make & specifications, maintenance history etc.

1.2 How the data is collected at present

The utility sends survey crews into the field. A crew walks a section of the network with a handheld GPS unit and records each asset it finds. For every asset the crew notes the code stencilled on the infrastructure/ pillar, its position, a condition rating from 0 to 10, and one or two measurements specific to that kind of asset - the height of a pole, the bore of a valve, the rating of a transformer etc.

At the end of the shift the handheld unit exports the day's work as a single CSV file. That file is emailed to head office. There, a data-entry operator opens it and re-types the contents into a master spreadsheet. The spreadsheet is the only record the utility has.

1.3 Problems with the present system

- Data reaches the office several days late, and re-typing adds new errors to the ones already present.
- The handheld units are old and their exports are untidy. Descriptions come with extra spaces and inconsistent capitalisation. Coordinates are sometimes written with a compass letter. Ratings are sometimes left blank.
- Crews occasionally record the same asset twice, or the unit writes a corrupt value into a field.
- When the operator meets a bad row, the usual response is to delete it. No record is kept, so genuine field observations are lost without anyone knowing.
- The spreadsheet cannot be queried. Answering a simple question, such as which assets in one area are in poor condition, means scrolling through it by hand.
- There is no access control. Anyone who can open the file can change or delete any row, and there is no record of who did.
- The utility company has commissioned a web map from a separate vendor. That map has no system to obtain data from.

1.4 The input data/ file supplied to you

You are given one file, data/survey_export.csv. It is a representative export from a handheld unit and contains 62 records. Following is the DS of the table:

Column	Contents
asset_id	The code stencilled on the asset, for example PL-0142. Format is two capital letters, a hyphen, four digits. Must be unique.
name	A free-text description written by the crew member.
asset_type	One of: pole, valve, manhole, transformer.
latitude	Position, north-south.
longitude	Position, east-west.
elevation_m	Height above sea level in metres. Often left blank.
surveyed_on	Date of the visit, in YYYY-MM-DD form.
surveyor	Name of the crew member.
status	active, decommissioned, proposed.
condition_score	A whole number from 0 to 10. Ten is as-new, zero is failed.
attribute_json	Measurements specific to the asset type, written as a small JSON snippet.

1.5 Known faults in the supplied file

The probability of a row containing a fault is approximately 1/6. These are the faults the utility sees in real exports, and your application must deal with all of them:

- Descriptions with leading, trailing or repeated spaces, and inconsistent capitalisation.
- Coordinates written with a compass letter, such as 28.6148 N, mixed in with plain numbers.
- Asset types written in different letter cases.
- A latitude outside the range a latitude can take.
- A longitude that is not a number.
- The same asset code on two different rows.
- A row with no asset code.
- An asset code that does not follow the utility's format.
- A condition rating of 14, on a scale that ends at 10.
- A condition rating left blank.
- A survey date in the future.
- An attribute snippet that is not valid JSON.
- An asset type the utility does not recognise.

2. Objective

Build a Python application that replaces the manual process described above.

The application must take the raw daily CSV from the field, convert it into clean and reliable asset records, store them permanently, and make them available over the network to authorised users and to the utility's web map.

2.1 Objectives in detail

1. Remove manual re-typing. The day's export is loaded by running one command.
2. Improve data quality. Untidy values are standardised automatically, unusable values are refused.
3. Stop silent data loss. Every record that is refused is written to a rejects file with the reason, so that a supervisor can review it.
4. Make the data query-able. Users can search and filter assets instead of scrolling a spreadsheet.
5. Keep the data safe. Records are held in a database, not a file, and survive a restart.
6. Control access. Users identify themselves, and only an administrator may delete.
7. Serve other systems. The utility's web map obtains its data from this application over the network.
8. Keep an audit trail. Every ingestion run and every request is logged.

2.2 Scope of your work

You are required to build the back-end only. There is no screen to design and no map to draw. Another group is building the map, it will connect to your application over the network.

The application has two parts, and both must be delivered:

- A command-line ingestion tool, run by a night-shift operator who does not know Python.
- A web service, used by staff and by the utility's web map.

3. Functional Scope for Development

The features listed below must all be built. They are numbered for reference during assessment.

3.1 Ingestion tool

1. The tool runs from a terminal in a single command, taking the path of the CSV file as an argument.
2. It accepts optional arguments for where to write the rejects file, the map file and the summary report.
3. It provides a help option that explains its usage.
4. It offers a strict option. In strict mode, any rejected row aborts the whole run. This is used when loading data that has already been cleaned.
5. Before processing, it confirms that the file contains every expected column. If a column is missing, the run stops at once and names what is missing.

6. On completion it prints a short summary to the terminal: rows read, rows accepted, rows rejected, and the location of the rejects file.

3.2 Data cleaning and standardisation

1. Descriptions are trimmed of surrounding spaces, repeated internal spaces are reduced to one, and the text is converted to a consistent capitalisation before storage.
2. Coordinates written with a compass letter are converted to plain decimal numbers. A southern or western reading becomes negative.
3. Asset types are matched regardless of the letter case used by the crew.
4. Surveyor names are standardised so that the same person typed three different ways is counted once.
5. The attribute snippet is read and its values made available as ordinary data.

3.3 Handling of rejected records

1. A row that cannot be trusted is set aside, and processing continues with the next row. One bad row must never stop the run.
2. Every rejected row is written to a separate rejects file, keeping all its original values and adding a column giving the reason for rejection.
3. No row is ever discarded silently.

3.4 Analysis and summary

1. For each asset type, report the number surveyed, the average condition rating, and the single asset in the worst condition.
2. Report the geographic extent of the survey, being the rectangle within which every accepted asset falls. The web map uses this to decide where to open.
3. List the assets that need repair, meaning those still in service whose condition rating is below 5.
4. Given a position, report which surveyed asset is nearest to it. Distance is the true distance over the curved surface of the earth, in kilometres.
5. List the distinct surveyors who worked on a given day.
6. Classify each asset's condition into a band: GOOD for 8 to 10, FAIR for 5 to 7, POOR for 3 to 4, CRITICAL for 0 to 2.

3.5 Output files

1. A map file containing every accepted asset as a geographic point with its remaining details attached, in the standard interchange format used by web mapping software.
2. A plain text summary report that a supervisor can print or paste into an email, containing a title, the time of the run, one aligned line per asset type, the survey extent, and the totals.
3. A rejects file as described in 3.3.
4. A log file to which every run appends one dated line, so that months later it can be seen which days were loaded and with what result.

3.6 Data storage

1. Accepted records are stored in a database. They are not held only in memory and must survive the application being restarted.
2. The asset code is unique in the database.
3. An asset is visited more than once during its life. The application keeps a history of visits against each asset, each visit carrying its own date, surveyor, condition rating and notes.
4. Deleting an asset also removes its visit history. No orphaned history is left behind.
5. The application can report the visit history of any single asset, and can report which assets have been visited most often.

3.7 Network operations on assets

The following operations must be available over the network. They are six distinct operations and the application must treat them as such.

1. List assets.
2. Fetch one asset by its code.
3. Add a new asset.
4. Replace an existing asset in full.
5. Correct selected fields of an existing asset, leaving the rest unchanged.
6. Remove an asset.

3.8 Search, filtering and paging

1. The asset list can be narrowed by asset type, by status, by surveyor, and by a range of condition ratings.
2. The asset list can be searched by a word appearing anywhere in the description, matched regardless of capitalisation.
3. The list is returned one page at a time. The caller states how many records it wants and from what position, and is told the total number available. A request must never attempt to return every asset at once.

3.9 Reporting over the network

1. A summary report giving the figures described in 3.4, in a form other software can read.
2. A list of assets needing repair.
3. A report of the most frequently visited assets.
4. A simple, unprotected status address that the utility's monitoring system can call every minute to confirm the application is running.

3.10 Bulk upload

1. A supervisor can upload a day's CSV file to the application over the network, instead of running the terminal tool, and is told how many rows were accepted and how many rejected.

3.11 User accounts and access control

1. A user signs in with a username and a password, and receives a credential which their software then presents on every later request.
2. There are two kinds of user. A surveyor may read, add and correct records. An administrator may additionally delete assets, upload bulk files, and create other users.
3. All asset data requires the caller to be signed in. The status address of 3.9 is the only exception.
4. An administrator can create further user accounts.

4. Non-Functional Scope

These are the qualities the finished application must have. They are as much a part of the assessment as the features in Section 3.

4.1 Validation rules

No record may enter storage without passing every rule below, whether it arrives from the CSV or over the network.

Field	Rule	If it fails
asset_id	Present, two capital letters, a hyphen, four digits, not already in use	Reject the record
name	Present, at least 3 and at most 120 characters after tidying	Reject the record
asset_type	One of the four recognised types, in any letter case	Reject the record
latitude	A number between -90 and 90	Reject the record
longitude	A number between -180 and 180	Reject the record
elevation_m	A number if present, may be blank	Store as not recorded
surveyed_on	A valid date, not later than today	Reject the record
status	One of: active, decommissioned, proposed	Reject the record
condition_score	A whole number between 0 and 10	Reject the record
attribute_json	Valid JSON	Reject the record

A further rule applies across fields: an asset marked decommissioned may not carry a condition rating above 2.

4.2 Error handling and reliability

1. A single unusable record must never stop a batch or bring down the application.
2. An attempt to create an asset whose code already exists is refused with an explanation, and the stored data is left exactly as it was.

3. A request for a record that does not exist receives a clear message saying so, not a system failure.
4. When a request is refused, the caller receives a consistent and readable explanation naming each field that was wrong and what was wrong with it. A raw framework error message is not acceptable to the utility's vendor.
5. The application distinguishes between five outcomes and reports each differently: a successful creation, a successful deletion, a request for something that does not exist, a request the caller is not permitted to make, and a request whose contents are invalid.
6. A caller who has not signed in and a caller who has signed in but lacks permission are two different situations and must be reported differently.

4.3 Efficiency and response time

Requirement	Target
A page of assets from the list operation	Within 500 milliseconds for a page of 25 records
Fetching one asset by its code	Within 200 milliseconds
The summary report	Within 100 milliseconds when served from cache
Maximum page size the caller may request	100 records, default 25
Ingestion of the supplied 62-row file	Within 5 seconds end to end

The web map requests the summary every few seconds and the figures are expensive to calculate. The summary must therefore be cached for up to 60 seconds. The cached figures must be discarded the moment any asset is added, corrected or removed, so that a stale summary is never served after a change.

To protect the application from a badly written client, any single caller is limited to 60 requests per minute. Beyond that the caller is refused politely and told when to try again. The status address is exempt from this limit.

4.4 Security

1. Passwords are never stored in a form from which the password can be recovered.
2. A password never appears in a response, a log entry or an error message.
3. The credential issued at sign-in expires after a set period and cannot be forged or altered by the holder.
4. No password, key or other secret appears anywhere in the source code or its history.
5. The web map is served from a different internet address to the application. The application permits that specific address to call it, and does not simply permit every address.

4.5 Configurability and portability

1. The database location, the signing secret, the credential lifetime, the permitted web addresses and the request limit are all supplied by the environment the application runs in. None is written into the code.
2. The application runs on a small database during development. The utility intends to move it to their enterprise database server before go-live. That move must be a change of setting, not a rewrite.
3. The same build must run unchanged on a developer laptop and on the utility's server.

4.6 Interoperability

1. The map file is written in the standard format used by web mapping software, so the utility's existing software can open it directly without conversion.
2. The application publishes its own documentation. The vendor building the map can visit an address in a browser, see every available operation with what it accepts and returns, and try it out without contacting us.

4.7 Logging and traceability

1. Every request is recorded with what was asked for, what was answered, and how long it took.
2. The caller is told in the response how long the request took.
3. Every ingestion run leaves a dated entry in a log that accumulates over time.
4. When the utility reports that the application felt slow at four o'clock, we must be able to answer them from the log.

4.8 Code quality, testing and documentation

1. The application is organised as a proper installable project, not a folder of loose scripts.
2. It runs in an isolated environment, and a colleague can reproduce that environment exactly from what is committed.
3. Automated tests are supplied that another engineer can run to satisfy themselves the application behaves. They cover the cleaning rules and their boundaries, the rejection of a bad row, each network operation, a request for something that does not exist, a successful sign-in, and a surveyor being refused a deletion.
4. Tests run against a throwaway database and never touch real data.
5. The repository contains instructions complete enough that an engineer who has never seen the project can go from a fresh copy to a running, populated application without asking a question.

5. Deliverables

1. A Git repository containing the source, the tests and the sample data, committed in steps rather than in one bulk commit.
2. A README explaining what the application is, how to set it up, how to run the ingestion tool, how to start the web service, how to run the tests, and what each network operation does.

3. A pinned dependency list, and an example configuration file showing what settings are required without revealing any real value.
4. The outputs of one ingestion run against the supplied file: the rejects file, the map file and the summary report.
5. A recording of five to eight minutes demonstrating the working application, to include the ingestion tool rejecting bad rows and continuing, the published documentation page, a sign-in followed by a successful request, a surveyor being refused a deletion, the summary served from cache and then correctly refreshed after a change, and a caller being refused for exceeding the request limit.